

A Secure Typst-Based Document Generation System for Modern Applications

Research Team
Computer Science
WAFT Research Lab
San Francisco, CA
research@waft.example

Abstract—This paper presents a secure Typst-based document generation system with comprehensive security hardening, efficient compilation, and unified template API. The system addresses limitations of traditional LaTeX-based approaches through modern design principles and security-first architecture.

Index Terms—Document Generation, Typst, Security, PDF, Templates

I. INTRODUCTION

Document generation is a critical component of many software systems, from academic publishing to business reporting. Traditional approaches based on LaTeX, while powerful, present challenges in terms of compilation speed, syntax complexity, and security considerations.

This paper presents a secure Typst-based document generation system that addresses these challenges through comprehensive security hardening, efficient compilation, and a unified template API.

II. RELATED WORK

Previous work in document generation has focused primarily on LaTeX-based systems. While LaTeX provides extensive capabilities, its multi-pass compilation model and complex syntax present limitations for modern applications.

Typst, introduced in 2023, offers a modern alternative with single-pass compilation and cleaner syntax. However, integration into production systems requires careful consideration of security and reliability.

III. SYSTEM ARCHITECTURE

A. TypstCompiler

The TypstCompiler provides secure compilation with multiple layers of protection:

- 1) Path Validation: All file paths are validated to prevent directory traversal attacks. Paths containing `..` are rejected, and absolute paths are restricted to allowed directories.
- 2) Content Size Limits: Default limit of 10MB prevents resource exhaustion. This is configurable per-instance.
- 3) Timeout Protection: Default 60-second timeout prevents hanging processes.
- 4) Subprocess Security: All subprocess calls use `shell=False` with list arguments.

B. Template Registry

The registry system provides auto-discovery of template wrappers through:

- 1) Module Scanning: Automatic discovery of Python modules in the wrappers directory.

- 2) Metadata Extraction: Extraction of template metadata from module docstrings.

- 3) Search Capabilities: Full-text search across names, descriptions, and tags.

C. Template Wrappers

Template wrappers provide a unified API for all templates, ensuring consistent behavior and error handling.

IV. SECURITY ANALYSIS

A. Threat Model

The system is designed to handle untrusted content, including user-provided Typst code and file paths.

B. Security Measures

Path validation prevents directory traversal. Content size limits prevent resource exhaustion. Timeout protection prevents denial of service. Subprocess security prevents command injection.

V. PERFORMANCE EVALUATION

Typst's single-pass compilation provides significant speed improvements over LaTeX. Typical compilation times are under one second for most documents.

VI. USE CASES

The system supports multiple use cases including academic publishing, business documents, and creative projects.

VII. CONCLUSION

The Typst-based document generation system successfully provides a secure, efficient, and developer-friendly alternative to traditional LaTeX-based approaches.